Zujajah Khan Khadijah Irfan

DEEPGUARD

Quick Submit

Quick Submit

Lahore Garrison University-Lahore.

Document Details

Submission ID
trn:oid:::1:3612791568

Submission Date
Jul 17, 2026, 8:24 PM GMT+5

Download Date
Jul 17, 2026, 8:29 PM GMT+5

File Name
SRS_DeepGuard_3.docx

File Size
7.6 MB

52 Pages

12,722 Words

71,975 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

DEEPGUARD: MULTIMODAL DETECTION OF MANIPULATED DIGITAL CONTENT USING TEXT AND IMAGE ANALYSIS

Software Requirement Specifications

Bachelor of Science in Software Engineering

By

	Name	Registration #	Mobile #	E-Mail
1	ZUJAJAH KHAN	SP-23/BSSE/031	03324713700	zujajah011@gmail.com
2	KHADIJAH IRFAN	SP-23/BSSE/025	03260802589	khadijahirfan025@gmail.com

Supervised by

Farhan Sarwar

Co-Supervised by

Khaleeq Ahmed

Department of Software Engineering
Lahore Garrison University
Lahore

Table of Contents

1. Introduction 5
1.1 Purpose 5
1.2 Document Conventions 5
1.3 Intended Audience and Reading Suggestions 5
1.4 Product Scope 5
2. Overall Description 6
2.1 Product Perspective 6
2.2 Product Functions 7
2.3 User Classes and Characteristics 8
2.4 Operating Environment 8
2.5 Design and Implementation Constraints ..... **Error! Bookmark not defined.**
2.6 User Documentation 9
2.7 Assumptions and Dependencies 9
3. External Interface Requirements 10
3.1 User Interfaces 10
3.2 Hardware Interfaces 10
3.3 Software Interfaces 10
3.4 Communications Interfaces 13
4. System Features ..... **Error! Bookmark not defined.**
4.1 Text Analysis 13
4.1.1 Description and Priority ..... **Error! Bookmark not defined.**
4.1.2 Stimulus/Response Sequence ..... **Error! Bookmark not defined.**
4.1.3 Functional Requirements ..... **Error! Bookmark not defined.**
4.2 Image Analysis 15
4.2.1 Description and Priority ..... **Error! Bookmark not defined.**
4.2.2 Stimulus/Response Sequences ..... **Error! Bookmark not defined.**
4.2.3 Functional Requirements 16
4.3 Context Consistency Checking ..... **Error! Bookmark not defined.**
4.3.1 Description and Priority ..... **Error! Bookmark not defined.**
4.3.2 Stimulus/Response Sequences ..... **Error! Bookmark not defined.**
4.3.3 Functional Requirements ..... **Error! Bookmark not defined.**
4.4 Sentimental Analysis 17
4.4.1 Description and Priority ..... **Error! Bookmark not defined.**
4.4.2 Stimulus/Response Sequences ..... **Error! Bookmark not defined.**

4.4.3 Functional Requirements 20
4.5 Metadata Analysis ..... **Error! Bookmark not defined.**
4.5.1 Description and Priority ..... **Error! Bookmark not defined.**
4.5.2 Stimulus/Response Sequences ..... **Error! Bookmark not defined.**
4.5.3 Functional Requirements ..... **Error! Bookmark not defined.**
4.6 Viral Pattern Detection ..... **Error! Bookmark not defined.**
4.6.1 Description and Priority 22
4.6.2 Stimulus/Response Sequences 23
4.6.3 Functional Requirements 23
5. Non-functional Requirements 24
5.1 Performance Requirements ..... **Error! Bookmark not defined.**
5.2 Safety Requirements ..... **Error! Bookmark not defined.**
5.3 Security Requirements ..... **Error! Bookmark not defined.**
5.6 System Constraints ..... **Error! Bookmark not defined.**
5.7 Assumptions and Dependencies ..... **Error! Bookmark not defined.**
5.8 Data Requirements ..... **Error! Bookmark not defined.**
5.9 Audit & Logging Requirements 30
5.10 Error Handling Requirements ..... **Error! Bookmark not defined.**
5.11 Business Rules ..... **Error! Bookmark not defined.**
5.12 Detailed Subsystem Design ..... **Error! Bookmark not defined.**
5.12.1 Text Analysis Subsystem ..... **Error! Bookmark not defined.**
5.12.2 Image Analysis Subsystem ..... **Error! Bookmark not defined.**
5.12.3 Context Consistency Subsystem ..... **Error! Bookmark not defined.**
5.12.4 Sentiment Analysis Subsystem ..... **Error! Bookmark not defined.**
5.12.5 Metadata Analysis Subsystem ..... **Error! Bookmark not defined.**
5.12.6 Viral Pattern Detection Subsystem ..... **Error! Bookmark not defined.**
5.12.7 Fusion Engine Subsystem ..... **Error! Bookmark not defined.**
5.13 System Design Diagrams ..... **Error! Bookmark not defined.**
5.13.1 Usecase Diagram 40
5.13.2 Erd Diagram 41
5.13.3 Architecture Diagram 43
5.13.4 Activity Diagram 44
5.13.5 Sequence Diagram 45
5.13.6 Component Diagram 46
5.13.7 State Machine Diagram 47
5.13.8 Class Diagram 47

5.13.9 Dataflow Diagram 48
5.13.10 Database Diagram 50
5.14 References 51

DeepGuard: Multimodal Detection of Manipulated Digital Content Using Text and Image Analysis

1. Introduction

1.1 Purpose

This document describes the functional and non-functional requirements of DeepGuard, a multimodal AI based system designed to detect fake and manipulated digital content on social media platforms. This document describes the scope of the system, including text analysis, image analysis, cross-modal consistency checking and web-based user interface.

This document (SRS) is intended for use by the development team, project supervisors, evaluators and stakeholders involved in the DeepGuard, Final Year Project. It provides a clear understanding of the system requirements, functionalities, constraints and design considerations, serving as a reference throughout the development, testing, and evaluation phases.

1.2 Document Conventions

This document follows IEEE SRS standard. The following conventions are used throughout:

- Bold text is used for terms being defined and requirement identifiers (e.g REQ-F-01)
- Shall denote a mandatory requirement, should denote a recommended but non-mandatory feature
- Functional requirements are prefixed as REQ-F and non-functional requirements as REQ-NF
- Higher level requirements are assumed to be inherited by their detailed sub-requirements unless stated otherwise

1.3 Intended Audience and Reading Suggestions

This document is for people.

• **Developers like Khadijah Irfan and Zujajah Khan:** They need to read the thing and pay close attention to the followimg sections 3, 4 and 5.

• **Project Supervisors:** It is an idea for the Project Supervisors to start with sections of 1.4 and 2 then look at the sections 4 and 5 to see what is technically required for this.

• **Testers and QA:** Testers and QA should look at the sectioon no.4 which's about System Features and section 5 which is, about the Non-Functional Requirements.

• **Academic Evaluators and Examiners:** Academic Evaluators and Examiners should read the document to get an idea of what it is about and really look at sections 2 4 and 5.6.

1.4 Product Scope

DeepGuard is a multimodal AI-based web application that detects fake and manipulated content on social media by jointly analyzing textual and visual inputs. The primary objective is to address the limitations of unimodal content analysis, which rely solely on either textual and visual information, by integrating Natural Language Processing (NLP) and Computer Vision (CV) techniques within a unified framework.

Key capabilities include: detection of misleading textual patterns using DistilBERT (advanced) and SVM with TF-IDF (baseline), detection of visually manipulated or AI-generated images using MobileNetV2 (transfer learning) and CNN baseline, cross-modal consistency checking to identify mismatches between text and image via Python logic comparison, sentiment analysis to flag emotionally manipulative content using keyword matching, viral pattern detection using Python Regex, and a confidence score prediction output with SHAP-based text explainability and GradCAM for image explainability.

The system is scoped as an academic prototype intended for local or cloud-based deployment. Real-time integration with live social media platforms is out of scope for this version.

2. Overall Description

2.1 Product Perspective

DeepGuard is a multimodal AI-based web application designed to detect fake and manipulated content by analyzing textual and visual information. It does not replace any existing institutional system but is positioned as a research prototype that extends the state of the art in multimodal fake content detection.

The system is designed as a standalone web application with a Streamlit-based frontend and a Python-based backend. It interacts with pre-trained machine learning models (DistilBERT and SVM+TF-IDF for text, CNN/YOLO and MobileNetV2 for images) and stores analyzed post history in a local SQLite database. The following diagram illustrates the major components:

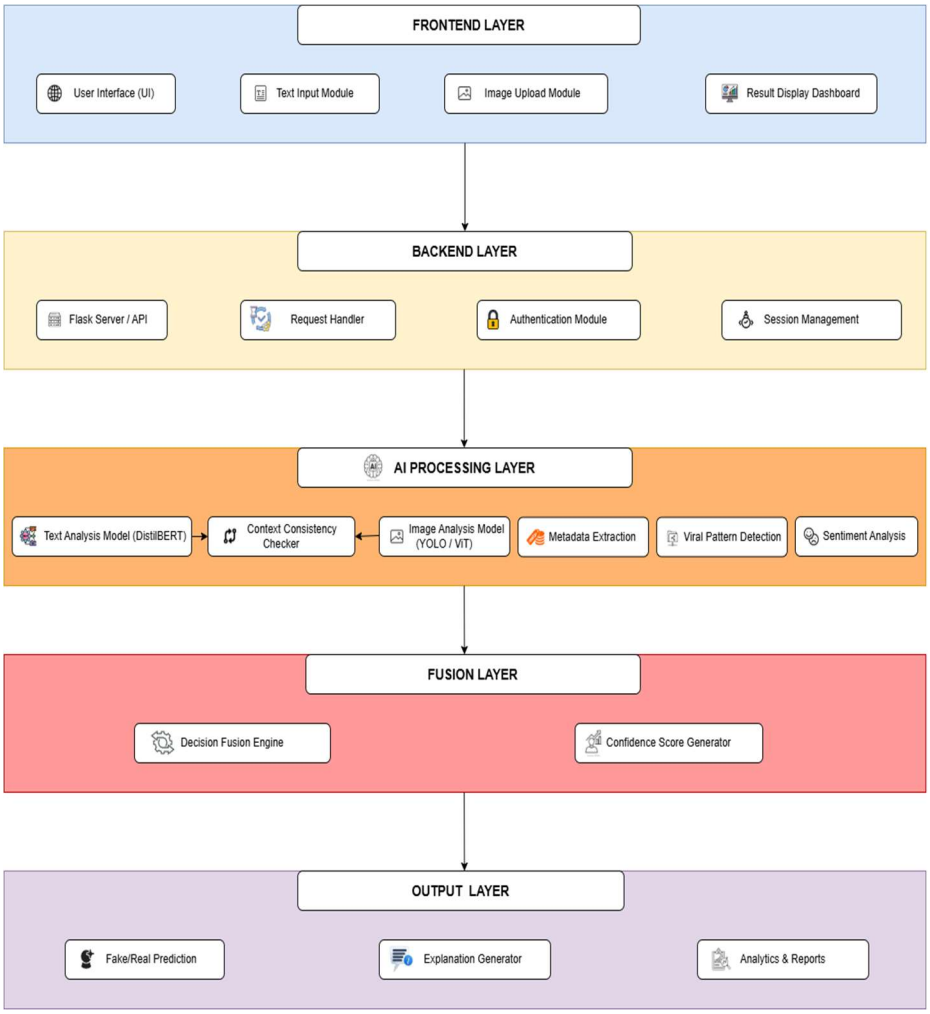


Figure 2.1. High-Level Architecture of the DeepGuard System

2.2 Product Functions

DeepGuard provides the following major functions:

- **Multimodal Input Processing:** Accept social media text and image simultaneously
- **Text Fake Detection:** Classify textual content as real or fake using DistilBERT fine-tuned on WELFake.
- **Image Authenticity Classification:** Detect whether an image is real or AI-generated or manipulated using MobileNetV2 (transfer training) trained on CIFAKE.
- **Cross-Modal Consistency Check:** Compare extracted keywords from text against image labels to flag contradictions.
- **Sentiment Analysis:** Detect emotionally charged or manipulative language patterns
- **Metadata Extraction:** Extract available metadata (file properties, EXIF data) using Pillow and ExifRead for supplementary context.
- **Prediction Output:** Generate a final verdict (Real/Fake) with confidence score (0-100%)

- **Explainability:** This HighlighTt influentiel wordz from the text that contributid to the classification decision of the system.

2.3 User Classes and Characteristics

Table 2.1. User Classes and Characteristics of the DeepGuard System

User Class	Description	Technical Level	Primary Use
General Social Media User	Individuals wanting to verify suspicious posts	Low	Submit content; view verdict
Journalist/Fact-Checker	Professionals requiring faster verification	Medium	Batch check and review confidence scores
Student / Researcher	Academic users studying misinformation	Medium-High	Analyze model outputs and explainability
System Administration	Manages deployment and monitors logs	High	Backend configuration and database access

2.4 Operating Environment

The DeepGuard System is made to work on the desktopp and laptoops computerzs that have at least 4 GB RAM. It is also better if the computer has a CUDA- GPU because it helps the system to learn and makee the decisions faster. The DeepGuard System works with Window10/11, Ubuntu and macOS 11 long as Python 3.10 is installed on the computer. People can use the system with web browsers like Google Chrome, Mozilla Firefox and Microsoft Edge.

The part of the system that users do not see is written in Python lamguage. It uses many tools like PyTorch, Hugging Face Transformers, OpenCV, Pillow, Flask, SHAP, Pandas, NumPy and Scikit-learn. The system uses SQLite to store the history of what it has learned and what it has predicted. The DeepGuard System learns mostly using Google Colab with a GPU to make it faster. When the system is readyy it can ran on a computer using the Flask development server or it can be put on any website that supporrts Python lamguage.

2.5 Design and Implementation Constraints

The DeepGuard System has to be written in thee Python 3.10 versiom for all the work it does that users do not see. The part that users see is made with HTML5, CSS3 and JavaScript. Its given to users through a Flask-based REST API. To make sure everyone can use the system and get the results it only uses open-source technologies and pre-trained models from places like Hugging Face Transformers and PyTorch. It does not need any APIs or services that cost the moneys.

The system can only use datasets that're available to the general public. For example it uses the WELFake dataset to classify text. The CIFAKE dataset to check if images are real or not. The system also explains its decisions using SHAP for text and Grad-CAM for images. This helps the people to understand how the system makes its decisions. The DeepGuard System is for the research and is not meant to be used in real-life situations like on social media or, in big companies.

Development is planned to be completed within a six-month academic timeline by a two-member development team. Model training is performed using the free GPU resources provided by Google Colab, and the project is developed without reliance on paid cloud infrastructure or commercial computing resources.

2.6 User Documentation

- **User Manual:** A PDF user guide explaining how to use the web interface, interpret results and understand confidence scores
- **README File:** A GitHub repository README covering installation, dependencies and usage instructions
- **Technical Report:** Full FYP documentation including system architecture, model evaluation and result analysis
- **Presentation Slides:** Slides prepared for the FYP defense covering project overview, methodology and results

2.7 Assumptions and Dependencies

The DeepGuard system assumes that users will provide at least one valid input for analysis, which may consist of textual content, an image, or both. The system further assumes that the required pre-trained model weights, including those for DistilBERT and MobileNetV2, are available locally after the initial setup or can be downloaded from Hugging Face Transformers during first-time

configuration. Trained baseline models and other learned parameters are expected to be stored locally after training using Google Colab.

The project depends on the availability of the publicly accessible WELFake and CIFAKE datasets for model training and evaluation. It also assumes continued access to free GPU resources provided by Google Colab throughout the development phase. During normal inference, the application operates independently without requiring continuous internet connectivity, except when downloading model weights during the initial setup or when optional updates are performed.

3. External Interface Requirements

3.1 User Interfaces

The DeepGuard user interface is implementted as a responsive web application designed to provide a simple and intuitive experience for both technical and non-technical users. The interface has a text box where you can type in your media post. Thiz text box can hold att least 500 characters. You can also upload a file, JPG, JPEG or PNG images as long as they are not bigger than 10 MB.

To start the anaalysis you just click the “Analyze button”. This button starts the system to check your post and image. Afterr the system finishes checking it shows you the results on a dashboard. The dashboard tells you if your post is Real or Fake how sure the system is and what features in the text made it decide that. It also shows the feeling of behind the post any information it found and if the text and image do not match.

This DeepGuard interface works well on computers and phones without needing any extra software. If you put in something, like a file that is not supported the system tells you what is wrong and how to fix it.

3.2 Hardware Interfaces

The DeepGuard system works on the computers with at least 4 GB of RAM. This helps it to run the models and the application. If your computer has a graphics card called a CUDA-compatible GPU the system uses it to work faster.

The system needs 2 GB of space to store the models, application files and a local database. This database keeps a history of what you have analyzed. You can upload images from your computer. The system does not need to connect to cameras or scanners. It only needs the parts of a computer to work and to process what you give it.

3.3 Software Interfacess

Table 3.1. DeepGuard Software Interfaces

Software Component	Version	Purpose	Interface Type
Python	3.9+	Core backend language	Runtime environment
PyTorch	2.0+	Deep learning framwork	Library API
HuggingFace Transformers	4.x	DistilBERT model loading and inference	Library API
OpenCV/Pillow	4.x/9.x	Image preprocessing and loading	Library API
Streamlit	1.x	Web UI framework	Web framework
SQLite	3.x	Local results storage	Embedded database
GitHub	N/A	Version control and collaboration	Code repository
Google Colab	N/A	GPU-based model training	Cloud platform
SHAP	0.x	Text explainability (token importance)	Library API
Scikit-learn	1.x	SVM model, TF-IDF, evaluation metrics (accuracy, F1, ROC)	Library API

Pandas / NumPy	2.x / 1.x	Data handling and numerical processing	Library API
Matplotlib / Seaborn	3.x / 0.x	Graphs, confusion matrix, ROC curve visualization	Library API
ExifRead / Pillow	2.x / 10.x	Metadata extraction from image files	Library API
Flask	3.10.11	REST API backend + HTML/CSS/JS frontend	REST API + Web framework
Kaggle	N/A	Source for WELFake and CIFAKE datasets	Dataset platform
VS Code	N/A	Primary IDE for development	Development tool
pip + requirements.txt	N/A	Package management and dependency tracking	Package manager

3.4 Communications Interfaces

The DeepGuard System usesz the HTTP/HTTPS protocol to talk to the frontend and the backend. The backend, which is built with Flaszk works on the default localhost port, which's 5000, when we are developing it. It also has endpoints that the frontend can use to send user inputs and get analysis results. This Fetch API is used for this. The application doz all the text and image processing on its own so it does not need to keep talking to services to do its reuired work.

We only need internet to set it up at first or to download pre-trained model weights from the Hugging Face Hub. After that the system works on its using models and resources that are stored locally. The DeepGuard System does not use any communication protocols or message formats so it works well with standard web technologies and is easy to deploy.

4. System Features

The DeepGuard system has features that work together to find fake content on social media. Each feature is a part of the system that helps with the analysis process.

The first feature is text analysis, which looks at the text that the user provides. The DeepGuard System uses the natural language processing techniques to understand what the text means and to find patterns and signs of the misinformation. The DeepGuard System checks if the text has any suspicious information or not insde it.

The second feture is image analysis, which looks at the images that the user do uploads. The DeepGuard System uses learning models to analyze the images and find any inconsestencies or manipulations. The DeepGuard System then says if the image is real or fake.

Another important feature of the DeepGuard System is context consistency checking, which compares the text and image inputs. The DeepGuard System checks if the image matches what the text says and finds any mismatches that might mean the informaation is misleading.

The DeepGuard System also has the sentiment analysis, which finds language in the text that is meant to do manipulate peoples emotions. This helps the DeepGuard System find content that is meant to scare people or make them react strongly which is often used in misinformation.

The DeepGuard System also has the Viral Pattern analysis, which finds patterns in content that are often used in misinformation that spreads quickly. This includes phrases that are shared a lot, language that triggers emotions and messages that try to create a sense of urgency. The DeepGuard System uses this feature to find content that might be manipulative or spread quickly on media.

The DeepGuard System also has a metadata analysis feature, which looks at the information like image properties and timestamps. This helps the DepGuard Sytem find content that's old or reused and presented as new.

Finally the DeepGuard System has a result generation feature that combines the results, from all the analysis componints. The DeepGuard System then gives a verdict, which says if the content is real or fake and gives a confidence score and a basic explanation of why the DeepGuard System made that decision//.

4.1 Text Analysis

The text analysis feature. Evaluates the text you provide in a social media post. It uses natural language processing to understand the organization and language patterns of your text. This feature looks for phrases, fake claims and common signs of misinformation like exaggerated statements, urgent langjuage and clickbait wogrds. Of just matching keywords it uses a pre-trained language model to examine the context of your text. It checks if your content seems genuine or possibly made-up based on what its learned from real news. The feature givez a score showing how likely your text is to be misleading. This score is used with other system features to make a predictcion.

4.1.1 Description and Priority

The text analysis feature checks the text of a media post to see if it has misleading or false information. It uses a model and natural language processing to look at thme meaning, language patterns and signs of misinformation. This includes charged language, exaggerated claims and clickbait phrasing. This function is crucial for spotting the content bassed on text alone. It offers a score that helps the system make a decision. This feature is very important because its a part of the DeepGuard system. Without text analysis the system can't evaluate or compare different types of inputs well.

Priority Rating

The Text Analysis feature is very benificial to the DeepGuard system with a score of 9. It helps detect fake textual content using advanced natural language processing techniques. The feature has a penalty score of 8 as poor text evaluation may reduce the reliability of the fake news detection. The implementation cost is estimated at 6 due to the integration of the models and computational requirements for the text processing. The risk score assigned is 7 as the features accuracy depends on the quality of the dataset, language variations aand the models capacity to accurately interpret semantic contexts.

4.1.2 Stimulus/Response Sequence

The text analysis feature starts when you enter text into the DeepGuard web application and initiate the analysis process. This action activates the text analysis module. The system preprocesses your text by removing characters and normalizing the input. The processed text is then sent to the natural language processing model, whinch examines its meaning, language patterns and signs of misinformation. The system assesses of the probability of your text being authentic or misleading. It Produces a classification score. The system generates a response with a text analysis result that features the predicted label and a confidence score. This outcome is sent to the result generation module, where its merged with outputs from features.

4.1.3 Functional Requirements

REQ-TA-01: The system shall allow you to provide input via the web interface for analysis.

REQ-TA-02: The system is required to check the input text for two conditions: that it is neither empty nor longer, than the permitted character count. Should it be invalid, the system will show a suitable error message.

REQ-TA-03: The system shall preprocess the input text by eliminating superfluous symbols, normalizing the text and preparing it for model assessment.

REQ-TA-04: The system will utilize thee natural language processing model to analyze the processed text, assessing its semantic meaning and identifying misleading patterns.

REQ-TA-05: The system shall identify the linguistic indicators of the misinformation, such as exaggerated claims, urgency-based language, and clickbait expressions.

REQ-TA-06: The system shall generate a classification score indicating the likelihood of the text being real or fake.

REQ-TA-07: The system shall provide the confidence score associated with the classification result.

REQ-TA-08: The system shall highlight important words or phrases in the text that contribute to the prediction, if explainability is enabled.

REQ-TA-09: The system shall pass the text analysis result to the result generation module for integration with other feature outputs.

REQ-TA-10: The system shall handle unexpected errors during processing and provide a user-friendly error message without system failure.

4.2 Image Analysis

The image analysis feature is responsible for examining the visual content uploaded by the user as part of a social media post. This functionality employs deep learning-based computer vision methods to analyze the image and determine whether it is authentic or it has been altered. It assesses visual patterns, discrepancies, and traits that could suggest image manipulation or synthetic creation.

The system processes the image using pre-trained models such as convolutional neural networks or vision-based architectures to extract meaningful features from the image. . The image analysis feature looks at the image. Checks if it is real or fake. It uses computer vision to find inconsistencies and editing marks. The image analysis functinnon examines the image to see if it is genuine or altered. It uses learning to identify patterns that might be hard to see annnd gives a score that shows the chance of manipulation. This feature is very immportant because it helps the DeepGuard system find misleading images.

Priority Rating

Benefit score given to it is 9**,** The Imagge Analysis feature is very helpful because it uses computer vision to detect fake images. Penaalty score given to it is 8, If the image analysis is not accurate it might not detect images and that would be bad. Implemmentation cost given to it is 7, It costs a lot to make this feature work because it needs a lot of commputer power and special technologies. Risk score iz 7 as there There is some risk because the feaaature depends on how good the model's the quality of the image and if it can detect sophisticated fake images.

Stimulus/Response Sequences

When a user uploads an image to DeepGuard the image analysias feature starts working. The system checks if the image is in the formmat and size. If it is okay the system prepares the image for analysis by resizing and normalizing it. The deep learning model then looks at the iimage and analyzes patterns and potential signs of manipulation. The system produces a response conssisting of an image analysis outcome, which features a classsaification label and a confidence score that reflects the probability of the image being authentic or altered. This outcome is then sent to the result generation module to be combined with other feature. Even if the user supplies solely an image without any text, the system conducts an image analysis and produces a partial result. Noneetheless, cross-modal analysis, like context consistency checking, is oomitted because there is no textual input. In these instances, the system shows that the analysis is th cconstrained because of inccomplete input.

4.2.3 Functional Requirements

REQ-IA-01: The user shall be able to upload an immage for analysis via the web interface, as permitted by the system.

REQ-IA-02: The uploaded image shall be validated by the system to ensure it is in a supported format (e.g., JPG, PNG) and does not exxceed the allowed size limit. Should it be invalid, the system will show a suitable error message.

REQ-IA-03: The system is required to preproccess the image by resizing, normalizing, and converting it into an appropriate format for model input.

REQ-IA-04: The system is required to utilize a deep learnning-based model to analyze the image for visual inconsistencies, manipulation artifacts, or synthetic characteristics.

REQ-IA-05: The system must produce a classifiication outcome that shows if the image is authentic or possibly manipulated.

REQ-IA-06: The system shall provide a connfidence score associated with the image classification result.

REQ-IA-07: The system shall pass the image analysis ressult to the result generation module for integration with other feature outputs.

REQ-IA-08: The system shall perform immage analysis even if no textual input is provided and generate a partial result.

REQ-IA-09: The system shall skip context connsistency checking if textual input is not available and indicate that the analysis is limited.

REQ-IA-10: The system must handle processing errors well so it shows a user- error message and does not crash.

4.3 Context Consistency Checking

The context coonsistency checking feature looks at how the text and image parts of a media post relate to each other. It checks if the text and image match. This function finds out if what the text says and what the image shows are consistent. It spots inconsistencies between text and image that might seem believable on their own but do not match when looked at together

The system uses nattural language processing to pull out information from the text like keywords, entities and claims. At the time it uses commputer vision to pull out semantic labels or visual cues from the image. The systtem then compares these to see if they match

If there is a mismmatch the system marks the content as misleading because of the inconsistency. If both inputs match the system considers the content contextually consistent. This feature is important for finding forms of missinformation that cannot be detected by analyzing text or image alone. This feature only works when both text and image inputs are prrovided. If either input is missing the system skips the context consistency checcking and proceeds with analysis using the available data

4.3.1. Priority

The context consisteency checking feature checks if the text and visual information in a social media post match. It compares elements from the text, like claimms, keywords and entities with semantic labels or visual cues from the image. The goall is to detect inconsistencies where both text and image might seem believable but do not match when analyzed together.

This feature is given priority because it helps the system recognize a crucial type of misinformation that independent text or image analysis does not effectively identify. By integrating -modal reasoning it improves the systems overall relliability.

Priority Rating:

The Context Conssistency Checking feature provides a benefit score of 9 because it enables the DeepGuard system to detect missleading content by analyzing the relationship between textual and visual information. This feature imprroves the reliability of news detection by identifying inconsistencies that may not be visibble through separate text or image analysis. The penaltyy score is 8 as the absence or inaccuracy of context consistency evaluation can significantly reduce the system's ability to detect -modal misinformation. Givenn the intricacy of merging language processing and computer vision methods for cross-modal compparison the estimated implementation cost is 7. The related risk score is 8 as the featuure relies on semantic interpretation, multimodal alignment and effective comparison of textual claims with visual content.

4.3.2 Stimulus/Response Sequences

The stimulus/response sequence for the context consistency checking feature starts after the user provides both image inputs and initiates the analysis process. This is the stimulus for activating the context consistency module. The systsem first receives processed outputs from the text analysis and image analysis modules. From the text side, key entities, keywords and semantic meaning are extracted, while from the of image side, visual labels or detected features are obtained. These outputs are prepared for comparison in a format. The system then performs a -modal comparison by the matching textual claims with visual representations. It evaluates whether both inputs refer to the event, object or the scenario. If alignment is found of the system marks the content as consistent. If contradictions or mismatches are detected the system flags the content as misleading due to contextual inconsistency. This feature's ultimate output is a consistency score. Where relevant a mismatch indicator. These are sent to the decision-making module for overall classification. This feature is executed when both text and image inputs are available; otherwise it is skipped.

4.3.3 Functional Requirements

REQ-CC-1: The system shall perform context consistency analysis when both text and image inputs are provided by the user.

REQ-CC-2: The system shall extract the entities, keywords and semantic meaning from the input text using NLP techniques.

REQ-CC-3: The system shall extract labeljjs or semantic features from the input image using a trained image analysis model.

REQ-CC-4: The system shall compare infornmation with extracted image features to determine semantic alignment.

REQ-CC-5: The system shall. Flag cases where the text and image are contextually inconsistent.

REQ-CC-6: The system shall generate a context consistency score indicating the level of agreement, between text and image.

REQ-CC-7: The system shall forward the comntext consistency score and mismatch indicator to the decision fusion module.

REQ-CC-8: The system shall skip context consistency anaalysis if either text or image input is missing and proceed with analysis.

REQ-CC-9: The system shall handle errors duuring feature extraction or comparison gracefully. Continue system execution without crashing.

4.4 Sentimental Analysis

The feature of semntiment analysis looks at how people feel when they read something on media. It checks if the words used are neutral happy, sad or try to manippulate peoples emotions. This is helpful in finding news that tries to scare or trickk people. The system uses methods to understand the words and how strong the emootions are. It looks for words that make people feel strongly phrases that use exaggeration and words thaat make people feel like they have to do something away. All these things are often used in misleading posts. This featurre helps figure out if something's safe or not.

This feature works on its own. Starts when theere is text to look at. If there is no text it does not do anything. The system keeps looking at other things.

4.4.1. Priority

The sentiment analysis feature checcks how people feel when they read something on social media. It sees if the words used are neutral happy, sad or try to manipulate peoples emotions. It looks for phrases that try to scare peopple make them feel like they have to do something away or use exaggeration. These things are often used in news.

This feature is important because it helplms find news that tries to manipulate peoples emotions. It is not the thing that decides if something is real or not but it helps. The Sentiment Analysis feature is given a score of 8 because it helps the DeepGuard system finmd news that tries to manipulate peoples emotions. It looks for words that try to scare people make them feel like they have to do something away or use exaggeration. The penalty score is 7 because if the sentiment anallysis is not accurate it can make it harder for the system to find news. The cost of implementing this feature is 5 because it is not as complicated, as things. The risk kscore is 6 because people can understand things depending on the context, sarcasm and how they talk.

4.4.2 Stimulus/Response Sequences

The sentiment analysis feature starts when someone puuts in some text and the system begins to analyze it. The text is what makezs the sentiment analysis feature start.

After it starts the system cleans up the text breaks it into pieces and makes sure everything is the same so it can be analyzed. The sentiment analysis feature looks at the text to see how the people feel when they read it. It checks the sentiment analysis of the text to see if it is neutral happy, sad or tries to manipulate peoples emotions. The sentniment analysis feature does this to help the DeepGuard system find news that tries to manipulate peoples emotions.. A naturral language processing component that assesses sentiment polarity and emotional intensity, then processes the test. The system detects emotional cues, incluuding words that induce fear, phrases that imply urgency, hyperboles, and manipulative language. It assingns a sentiment classification as neutral, positive, negative, or highly emotional/misleading based on the analysis. The output is converted into a sentiment score and sent to the final decinsion fusion module, where it helps inform the overall risk assessment of the content. If no tekxt input is provided, the sentiment analysis module remains inactive, and no response is generated from this feature.

4.4.3 Functional Requirements

REQ-SA-1: The system shall conduct sentiment analysis solely upon receiving textual input from the user.

REQ-SA-2: Prior to analysis, the system shall preprocess the input text by eliminating noise, correcting punctuation inconsistencies, and removing irrelevant characters.

REQ-SA-3: The system will examine the text to identify sentiment polarity, including positive, negative or neutral.

REQ-SA-4: The system must identify emotionally charged and manipulative language, including urgency, fear, or exaggeration.

REQ-SA-5: The system must give a score that shows how strong the feelings are in a piece of text.

REQ-SA-6: The system will give a result that says if the sentiment is negative or neutral and a score that shows how sure it is.

REQ-SA-7: The system will send the results of the sentiment analysis to the decision module to make a prediction.

REQ-SA-8: If there is no text the system will skip the sentiment analysis. Keep working without any problems.

REQ-SA-9: The system must handle text that's invalid or empty properly and give an error message or no response.

4.5 Metadata Analysis

Metadata analysis lookks at hidden information in image files. This includes when the image was made changes made to it the device used and software used for editing.

The goal is to find inconsistenciies that could mean the image was manipulated or used in a way.

The system processes uploaded images to get metaadata like EXIF data. It checks this information for anomalies like missing metaadata or signs of the editing software. These signs help to evaluate how teh genuine the image is and where it came from. Metadata analysis works on its own from text input. Happens whenever an image is sujpplied. If metadata is not avanilable the system notes this. Continues analysis using other features.

4.5.1 Description and Priority

The metadata analysis tool looks at hidden information in an image file called EXIF data. This includes specifics like when the image was made the device model and sooftware used for editing. The feature helps recognize inconnsistencies that could suggest image alteration or inappropriate use. This feature is given priority because it provides supporting evidence rather than direct classification. It helps strengthen detection accuracy but cannot independently determine if content is fake or real.

Priority Rating:

The Metadata Analysis feature is a provnides a benefit score of 7 by supporting image authenticity verification through the following as u know the EXIF and hidden filez the information analysis. The penalty score is 6, as missing or inaccurate metadata evaluation may reduce detection support.

4.5.2 Stimulus/Response Sequences

An image triggers the metadata extraction module. The system retrieves the image file. Tries to extract metadata, including EXIF information. It obtains details like timestamp, device model and

software signatures. The extracted metadata is examined for anomalies like missing metadata or signs of editing tools. These findings do the evaluate if the image has been modified or is missing origin details. The final oumtput is an analysis report or score sent to the fusion module for final decision-making. If there is no image or metadatma the system notes this limitation. Proceeds with other features.

4.5.3 Functional Requirements

REQ-MA-1: The system will extract mmetadata from images using EXIF.

REQ-MA-2: The system will identify mettadata fields like creation date and device information.

REQ-MA-3: The system will analyze metadata to detect inconsistencies or suspicious modifications.

REQ-MA-4: The system will generate an analysis score indicating authenticity or suspicion.

REQ-MA-5: The system will flag images with missing or tammpered metadata as suspicious.

REQ-MA-6: The system will forwarrd metadata analysis results to the decision fusion module.

REQ-MA-7: The system will continue execution even if metaadata is unavailable or extraction fails.

REQ-MA-8: The system will handle corrupted image files. Display an error message.

4.6 Viral Pattern Detection

Viral pattern detection aims to recognize structures in misleading social media posts. This includes chain messages, hyperbolic assertions and repetitive forwarding behaviors. The sysstem examines text using pattern recognition methods. It iddentifies keywords formatting styles and structural indicators, like " this" or "breaking news" patterns often linked to viral misinformation.

This feature works independently of image input and contrributes to the overall risk assessment by identifying whether the post follows known misinfoormation propagation patterns. In the absence of text, the module is bypassed, and the system proceeds with other available features.

4.6.1 Description and Priority

The viral pattern detection feature identifies structural and linguistic patterns commonly found in misinformation and chain messages circulating on social media platforms. It focuses on detecting repetitive forwarding behavior indicators, eexaggerated claims, clickbait phrases, urgency-based wording, and typical fajke news dissemination formats such as “forward this message,” “share immediately,” or “breaking news” styled posts.

This feature has been given medium priority duee to its provision of robust heuristic signals for identifying potentially misleading content, particcularly in the initial stages of detection. It is rule-bassed and cannot fully grasp semantic meaning, so it seerves as support for, but does not replace, core macchine learning-based modules.

Priority Rating:

The Viral Pattern Detection feature provides a benefit score of 7 by helping identify common misinformationnn structuures such as clickbait phrases, chain messages, and urgency-based content. The penalty score is 6, as weak pattern deetection may reduce early misinformation identification. The implementation cost is 4 bevcause it mainly relies on rule-based techniques, while the risk score is 5 since pattern-based detectioon may not always accurately interpret content meaning.

4.6.2 Stimulus/Response Sequences

The stimulus/response sequence for the viral pattern detection feature begins when the user provides textual input and initiates the aanalysis process. Text serves as the impetus for triggering the viral pattern detection module.

After being triggered, the system preprocesses the input text by normalizing the case, eliminating superfluous symbols, and oorganizing the content for paattern matching. The system subsequently employs rule-based verifications and regularr expression patterns to detect established signs of viral misinformation.

These patterns encompass urgency-driven phrases, repetitive forwarding instructions, hyperbolic claims, and clickbait-style expressions that are ooften found in fake or misleading posts. The system assesses how often and how strongly these patterns occur in the text.

The output is a viral pattern score indicating the likelihood that the content follows misinformation dissemination structures. This result is then fowarded to the final fusion module. When there is no text input, the module does not activte and no analysis occurs.

4.6.3 Functional Requirements

REQ-VP-1: The seystem is to analyze textual inpunt to identiffy established viral and chain message patterns that are typically linked to misinformation.

REQ-VP-2: The systm is required to preprocess text by normmalizing the case, eliminating superfluous symbols, and organizing input for pattern recognition.

REQ-VP-3: The systm is required to employ rule-based metehods and regular expjnressions to detect viral indicators, including urgency phrases and forwarding instructions.

REQ-VP-4: The system shall detect clickbait-style expresnsions and exaggerated claimsm commonly used in misleading posts.

REQ-VP-5: The system shall gmnerate a viral pattern score indicating the likelihood of misinformation propagation style.

REQ-VP-6: The systemm shall forward the viral pattern score to the final decision fusion module.

REQ-VP-7: In the absence of a text input, thew system shall bypass viral pattern detection and carry on executing without any interruptions.

REQ-VP-8: The system should manage invalidd or empty text input gracefully, providing a suitable null or error response.

5. Non-functional Requirements

DeepGuard's goal is to provide more accurate, reliable and efficient multi-modal detection of fake content without any compromising user experience. The system should also deliver analysis results within a realistics time frame; even if multiple AI modules are executed concurrently. It is important that there is a high level of prediction accuracy and reliability to ensure reliable detection of misleading content. The architecture should be scaelable and accommmodate future growth and increasing user demands. The interface ought to stay straightforward to use, enabling usersz to upload text and images with ease and without technical complications. It should also include proper error handling, secure data processing, and uploading data security too ensure the stability of the system and the integrity of the data. There are a number of non-functional requirements listed below:

5.1 Performance Requirements

The DeepGuard seystem must perform the following to enable efficient and timely software analysis under normal and peak operating loads:

The time it takes for a file to be uploaded to the server.

Under normal conditions, the DeepGuard System shall validate and process any user uploaded textual and image inputs in 5 second. After validation of the inputs has been accomplished, the system shall start the multimodal analysis pipeline and produce preliminary processing results in 10-20 seconds for typical text and images input. Response times can differ based on input volume, system load and hardware acceleration available.

5.1.2 Analysis Processing Time

The system shall analyze typical user submissions within 10-20 seconds of use in normal conditions. The processing time is based on the selected analysis modules, the size of the uploaded image, the length of the text input and the computing power available. GPU accelerations systems may provide faster inference times compared to CPU-only seystems.

5.1.3 Report Generation Time

The system shall produce and present the the final prediction, confidence score, explainability results and supporting analysis details within 5 seconds, upon successful completion of the analysis process.

The result generated should be displayed to the user via the user interface without further processing by the user.

5.1.4 System Load Handling

The DeepGuard System shall be able to facilitate a minimum of 50 concurrent userz at no significant loss of performance. Under nominal load, the average response time should not exceed 5 second; under moderate load, the system shall not lose the ability to process user requests reliably without interruption or data loss, and the average response time shall not exceed 5 seconds.

5.1.5 File Handling Capacity

It will handle text of up to 500 characters and images up to 10MB in JPG, JPEG and PNG. Uploaded files shall be checked before performing any processing and if the file size or type is not supported by the application, a user-friendly error message shall be returned.

5.1.6 Availability and Uptime

In normal use, the operational availability of the DeepGuard seystem shall be about 99% without counting scheduled maintenance. Planned maintenance work should not exceed 2–4 hours per month and should be maintained in a consistent manner, allowing the application to be accessible at all times.

5.2 Safety Requirements

DeepGuard systemm is built to be an analysis of software and find vulnerabilities. Although it is not a life-critical system, safety requirements are set to ensure that the system is not misused, data potentially lost or analysis results were incorrect, thereby avoiding security problems.

5.2.1 Prevention of Harmful Code Execution

All of the data entered by the user shall be checked for correctness prior to processing. All text submissions will be reviewed to verify that they are not blank, and that they adhere to the length limitations of the supported formats; uploaded image files will be reviewed to verify that the file is not too large and supported formats are used. Input that is invalid or corrupted shall be rejected and shall return an appropriate and meaningful error message to avoid unnecessary processing and to keep the system stable.

5.2.2 Data Loss Prevention

All uploaded image will be processed for as long as the analysis is processed, unless the user selects to save the results in the analysis history. To reduce storage requirementz and the potential for

unauthorized access, temporary files generated during processing shall be automatically removed after processing.

5.2.3 Protection from Malicious Input.

The system shall provide monitoring of the use of computational resources (CPU, memory, storage) to ensure the stable operation of the system during the analysis. Resource-intensive operations shall be handled in an efficient manner to avoid conzuming too many resources that may adversely impact the system's performance or availability.

5.2.4 System Failure Handling

If the system crashes or fails during analysis, it should be able to stop running processez that were already running, protect the data that is already processed and tell the users that the analysis has not been completed.

5.2.5 User Data Safety

The system shall provide clear, user-friendly error messages for operations that cannot be carried out and explain what is required. Error messagez shall be descriptive without revealing sensitive implementation details, paths within the files, model configuration etc. or other information that may lead to security risks.

5.2.6 Error Prevention Measures

All trusted and actively maintained third party libraries and frameworks shall be used for the DeepGuard system. All dependencies must come from official sources, be monitored for security updates and updated periodically to reduce vulnerability and ensure system compatibility and stability.

5.3 Security Requirements

Users uploaded software and analysis reports are managed by the DeepGuard system, so proper security and privacy protection shall be implemented to maintain system integrity, user data and analysis results.

5.3.1 User Authentication

The system shall prommpt the user to log in with a valid user name or email address and password before accessing facilities that are protected such as analysis history and user profile information. No passwords will ever be stored plainly and will be securely hashed before being stored in the database.

5.3.2 Authorization & Access Control

The system shall restrict authenticated users from accessing and viewing content that is not theirs, or analysis history that is not theirs. Unauthorized access to the information of other users will be automatically refused.

5.3.3 Data Privacy Protection

Any information or information entered by users, images uploaded, or any text information entered into the system, or any analysis records, shall be protected from unauthorized access or alteration within the system. All sensitive information will be treated securely when stored and processed, ensuring users' privacy and data integrity.

5.3.4 Secure File Handling

In a production environment, communication between the client and the server shall be secured by the HTTPS protocol to ensure that it cannot be intercepted by other parties or modified. Communication can be done via HTTP to the local environment during local development.

The prevention of injection and validation of input must be secured.

The system shall keep user sessions secure following successful authentication and shall end user sessions when users log out. No invalid or expired sessions shall be allowed to access resources of the protected system.

5.3.6 Session Security

The application should provide input validation and security in the programming, which will minimize the threat of common web application attacks such as SQL injection, cross site scripting (XSS) and malicious file uploads. Uploaded files should be checked prior to processing and any unexpected or unsupported input should be safely denied.

5.3.7 Secure Communication

The client and the DeepGuard server shall communicate securely by using secure communication protocols as HTTPS when the system is deployed in production environment. All data shall be encrypted while being transmitted between the client and the server to prevent interception, unauthorised access or tampering during the transmission. The use of localhost:http is acceptable for local development environments to test.

5.3.8 Audit and Logging

The system shall keep records of user log-ins, files uploaded, analysis requests and other important system events. Audit logs will be in a manner that prevents unauthorized modification, deletion, and will be accessible to authorized users for monitoring, troubleshooting, and security auditing purposes.

5.3.9 Requirements for Compliance

The DeepGuard System should follow the cybersecurity best practices and maintain confidentiality, integrity, and availability of data, which is also known as the CIA triad. The system, if used within an institutional or enterprise setting, shall also meet the requirements of the organization's data protection policies, cybersecurity standards and relevant regulations.

5.3.10 Security Measures to Prevent Unauthorized Use of Information Technology Systems.

The system shall identify repeated unauthorized attempts to logon to the system and limit or deny login access to the system for a period of time after a period of repeated unsuccessful attempts to logon to the system. Access to the backend services, application resources and SQLite database shall be controlled and authorized by an appropriate authentication mechanism and access control mechanism for the system administrators and users.

5.4 Software Quality Attributes

The DeepGuard System should meet the following qualities to operate efficientlly, reliably and securely for users who analyze software and discover vulnerabillities.

5.4.1 Usability

The DeepGuard system shall have an easy to use, intuitive and responsive user interface for technical and non-technical users. New users are able to complete primary functions (such as logging in and submitting text or images for analysis and interpretation of prediction results) within 5 – 10 minutes of their initial use. The experience of the user shall be optimized by reducing of the unnecessary complexity while providing the results of the analysis in a clear and understandable way.

5.4.2 Reliability

The system shall be stable and consistent in operation and be able to complete at least 95% of the valid analysis requests during normal operation. If an unforeseen failure should occur, the system shall ensure that no corruption of users data occurs, nor shall any previously generated analysis results be corrupted.

5.4.3 Availability

DeepGuard system, shall operate at about 99% availability during normal operational hours (excluding scheduled maintenance). The maintenance activities should be no more than 2 to 4 hour per month, to ensure that users can access it at all times.

5.4.4. Correctness

The system shall provide analysis results that reflect the probability that the provided content is authentic, manipulated, or AI-generated, according to the output of the machine learning model and analysis modules implemented. The false positive and false negative rates shall be reduced as much as possible to ensure detection performance is reliable for academic evaluation.

5.4.5 Performance Efficiency

The system shall be responsive during normal uses and start serving requests within 2-3 seconds after the request is submitted. A typical multimodal analysis shall be carried out within the performance limits specified in Section 5.1, depending on the size of the textual input, image resolution, and computational resources available.

5.4.6 Security

The system shall ensure the confidentiality, integrity and availability of data belongings to the user by means of secure authentication, authorization, and access control mechanisms. Unauthorized accesses shall be identified, limited and logged in the system audit logs.

5.4.7 Maintainability

The DeepGuard System shall be structured in modular software components allowing for individual analysis modules, Fussion Engine, user interface and database components to be updated, tested or replaced without interfering with the remaining part of the system to a significant degree. The coding standards will be used to make the source code easy to maintain, debug, and improve over time.

5.4.8 Portability

The system shalls be able to be deployed on multiple operating ssystems, such as Windows, Linux, and macOS, and it shall be deployable on standard web hosting that is capable of running Python. Minimal configuration changes should be needed to migrate between supported platforms.

5.4.9 Scalability

The system shall be able to handle more users and more volumez of analysis requests, without performance degradation, as the number of users grows. The architecture will also provide for future

scalability, allowing for the addition of new analysis modules, more sophisticated machine learning models, and enhanced detection capabilities.

5.4.10 Testability

The system will be designed in modular componentss where each module can be unit tested, integrated tested and system tested independently. Individual modules (authentication, text analysis, image analysis, Fusion Engine, and result generation) will be testable without the need to alter unrelated modules.

5.4.11 Flexibility

Future improvements to the DeepGuard system will be implemented without significant redesign of the system, by incorporating more machine learning models, analysis techniques, explainability methods and detection featuress. System configuration and analysis logic should be flexible with limited impact on the existing functionality.

5.4.12 Interoperability

The system shall accept standard text input and commonly used image formats, such as JPG, JPEG and PNG, to enable the system to be compatible with various platforms and devices. The architecture shall also be designed to allow for integration with external APIs, datasets, or verification services, as needed, while miniimizing the impact on the underlying system.

5.6 System Constraints

The DeepGuard System is subject to various technical, operating and environmental limitations that set limits on the operation and applicability of the application. These constraints help to make the system compatible, reliable, and consistent during development and deployment.

5.6.1 Platform Constraints

The DeepGuard Systemm will be a web-based software application that will be available in modern web browsers. The application should be compatible with commonly used browsers, such as Gooogle Chrome, Mozilla Firefox, and Microsoft Edge, without the need for any extra browser plugins or extensions.

5.6.2 Technology Constraints

The system is to be developed with Python 3.10 for backend processing, Flask as a web application development framework and HTML5, CSS3 and Javascript for the frontend. The machine learning models shall be realized with PyTorch, Hugging Face Transformers, OpenCV, and other open source

libraries approved. For the core detection capabiilities, the system shall not rely on proprietary software or commercial APIs.

5.6.3 File Size Constraints

The system should be able to receive user input through text and upload of images. Images will be no larger than 10 MB and will be submitted in JPG, JPeG or PNG. Text submission will follow the maximum input length supported in the system and any text file not supported or larger than the maximum will be returned along with appropriate validation messages.

5.6.4 Performance Constraints

The performance of the system shalls be based on the hardware used, such as CPU, memory, and optional GPU acceleration. For larger images or more complex analyses, the system may take longer to process, but will not exceed the performance limits specified in Section 5.1 under normal operating conditions.

5.6.5 User Environment Constraints

Users will be required to have devices capable of supporting current web browsers with javaScript enabled. The web application will require internet connections to access the web and, if applicable, to download pre-trained model weights when the system is first set-up.

5.6.6 Database Constraints

The system shall employ SQLite as a database for user accounts, authentication details, submission history, analysis results, and other related mettadata. The database shall be able to be accessed concurrently by more than one user without losses of reliability for the expected workload and without affecting the integrity and consistency of the data.

5.6.7 Security Constraints

Access to protected features – such as analysis history and account information – shall be denied without user authentication. All password are to be stored only in a securely hashed form and access to any resources in the back end and to the database is to be limited to those who are authorized.

5.6.8 Regulatory Constraints

The DeepGuard Ssystem shall process and store user data in compliance with universally accepted data protection, privacy and cybersecurity principles. In an institutional or enterprise setting, the system will also be compliant with policies and regulations.

5.7 Assumptions and Dependencies

The DeepGuard System makes a number of assumptions and has a number of external dependencies that it must have to function correctly during developments and deployment. The following assumptions provide guidance on the operating conditions for the system and are necessary to ensure that the system meets the intended operations within the academic scope.

5.7.1 User Assumptions

This system assumes some basic knowledge of using web applications and is able to interact with the interface to register and logs in and to submit text and/or image inputs for analysis. It is also assumed that users will supply accurate registration data and provide content that is suitable for analysis and in compliance with the input format supported and system requirements.

5.7.2 Input Assumptions

The system assumes that text that is submitted and images that are uploaded are not intentionally corrupted or malicious beyond normal testing situations. The input datta should meet the validation criteria specified by the application to enable successful analysis by the analysis modules.

5.7.3 System Environment Assumptions

The system assumes that all users will have access to the app via a current, JavaScript capable web browser and that there is a stable internet connection for the user to access the web application. It is also possible that internet connection will be needed for initial setup, to download pre-trained model weights and other dependencies.

It is essential that the hardware and software requirements are met.

The DeepGuard system relies on a computer system capable of running machine learning inference with the Python language and software libraries, such as PyTorch, Hugging Face Transformers, OpenCV, Flask, Pillow, SHAP, Pandas, NumPy, and Scikit-learn. Persistent storage of user accounts, user submission, and the history of analysis also relies on SQLite.

5.7.5 Network Dependencies

When using the normal Web application, the system uses network connectivity to send and receive information between the client and the server. A network connection is also needed when users post their content for analysis, dowmnload prediction results, and download pre-trained model weights as part of the initial installation or during updates.

5.7.6 Third Party Dependencies

Its core functions rely on trusted open source frameworks, libraries and publicly available datasets. They commprise Hugging Face Transformers, PyTorch, OpenCV, the WELFake dataset for text analysis and the CIFAKE dataset for image authenticity detection. The system shall continue to be compatible with versionz of these dependencies which have been supported for reliable operation and reproducible results.

5.8 Data Requirements

The data for user authentication, content analysis, data predicction history and system operation shall be safely stored, managed and maintained by the DeepGuard System. Confidentiality, integrity, availability and controlled access of the data in relation to its entire lifecycle shall be ensured through appropriate measures.

5.8.1 User Data Storage

The system shall maintain data for user registration, such as user name, e-mail address, and securely hashed passwords. Personal information will not be exposed to unauthorized access, and password will not be recorded in plain-text.

5.8.2 Authentication Data

It shall provide authenticated user sessions, including storage of information for managing active authenticated sessions, session validity, and session expiration. The session information will be kept secure to avoid any unauthorized access or hiijacking of the session.

5.8.3 Uploaded Content Data

The system shall preserve information about content submitted by users such as file name, file type, file size, time of upload and any other information necessary to support system operation. Images will be uploaded and processed securely throughout the analysis process and temporary files will be deleted or archived per storage policy of the system after processing.

5.8.4 Analysis Data

The system shall keep analysis records of every submitted request with the final prediction, the confidance score, the sentiment analysis, the semantics and metadata results, the consistency of the context and the explainability results and the time the the analysis was performed. Each analysis record should be tied to the authentication user that was used to create it, to facilitate secure access to past analysis history.

5.8.5 Analysis History

The system shall be capable of keeping a structured history record of submitted analyses and results of the prediction thereof, and of allowing authenticated users to view their past analyses and prediction results. Analysis history to be organized efficiently for ease of retrieval, viewing, and future reference.

5.8.6 Data Integrity

The system shall maintain the integrity, completeness and security of user information stored within the system, information attached to content uploaded to the system and analysis records. Content submitted to the system shall be consistent with the corresponding analysis and analysis results shall be appropriately validated and integrated to the database.

5.8.7 Data Retention

The system shall have a data retention policy to determine the lifetime of user information, uploaded content metadata and analysis records. The temporary processing files shall be deleted automatically following analysis, except for analysis history purposes. Also, the system shall provide the ability to delete user data in compliance with system policies or by the authorisation of users.

5.8.8 Data Security

All data stored shall be protected from unauthorized access using appropriate authentication, authorization and secure storage mechanisms. All sensitive information, such as user authentication data and user credentials, shall be stored seccurely and with industry standard security practices in place, ensuring confidentiality and integrity.

The following requirements are recommended for auditing and logging:

The DeepGuard ssystem shall record extensive audit logs for security monitoring, troubleshooting, performance analysis and accountability. Logging mechanisms must be able to capture key events of the users and systems and ensure that log information is not accessed or modified by unauthorized partiess

5.9.1 User Activity Logging

The system shall capture important user activities such as: user registration, user sign-in, user sign-out, content upload, profile updates, analysis requests. Every recorded event should contain the correct date and time and the user account who recorded the event to help with auditing and activity tracking.

5.9.2 Authentication Logs

The systems shall keep a detailed record of all successful and unsuccessful authentication attempts. Attempts to log in that fail shall be tracked and logged for purposes of security analysis, intrusion detection and account protection.

5.9.3 Analysis Activity Logs

The system shall document all analysis requests by the user, whether or not they are textual, image, or both. All log entries will contain the time of submission, time of analysis, processing status, predictions made and other information about the system that might be useful for monitoring and troubleshoooting.

5.9.4 System Event Logs

The system shall keep a record of important system events such as application errors, unexpected exceptions, processing failures and system crashes. Record performance related events and resource utilization problems shall also be recorded to help debug, maintain and optimise performance.

5.9.5 Security Logs

The system shall document unauthorized access attempts, denied operations, suspicious user activity or other security-related events. Security logs should facilitate monitoring and investigation of security incidents and be used for identification of potential threat to the application.

5.9.6 Data Access Logs

Access to sensitive data (such as user accounts, authentication data, analysis history and prediction results stored) shall be recorded by the system. Access records shall be kept so that only authorized users are able to acceess or modify protected information and support auditing of access activities to protected information.

5.9.7 Log Storage and Protection

Audit logs shall be maintained securely and shall not be modified, deleted or discllosed, without authorization. Log access should be limited to authorized administrators or other users with adequate privileges to maintain the integrity and confidentiality of audits logs.

5.9.8 Log Retention

The system shall maintain audit logs for a certain period as determined by the application's storage policy and its operational needs. Upon expiration of the retention period, logs will be retained in

accordance with a defined data retention policy, with information retained in accordance with the needs for security investigations or regulatory compliance.

5.10 Error Handling Requirements

The DeepGuard System will have detailed error handling procedures to guarantee reliable operation, data integrity and meaningful feedback to the user in case of unforeseen situations. Errors will be caught, dealt with gracefully and entered into a log to be used for troubleshooting, but will not affect system stability.

5.10.1 Input Validation Errors

All input must be validated prior to processing. Any submission to this form that is missing text, is a non-supported image type, has an oversized file, or fails to be formatted properly, shall be rejected with a clear and user-friendly error message that provides explanation of the cause of validation failure.

5.10.2 Authentication Errors

The system shall inform the users if they enter invalid login information. If the user enters multiple consecutive wrong passwords, then he will be subject to appropriate protection measures from the account, such as imposing a temporary ban on access or limiting access rates, which all aim to prevent illegal access.

5.10.3 File Upload Errors

The system shall check uploaded images prior to analysis and shall alert users if an unsupported file format or an oversized file is uploaded. No invalid uploads will be processed and clear information will be provided to the usser on the required format of upload and size restrictions.

5.10.4 Analysis Failure Handling

The system shall be able to recognize and respond to failures in the text, image or multimodal analysis without unpredictable termination. When an analysis is not successfully completed the system shall inform the user with a meaningful error message and shall retain any data that was previouslly stored and shall maintain overall system stability.

5.10.5 System Failure Handling

The system shall identify unexpected application failurez and stop any affected processes in a safe manner, preventing corruption of the information stored. Where possible, user data and the results of the analysis shall be stored to maintain the integrity of the data and aid in data recovery.

5.10.6 Network Error Handling

The system shall alert the user if the network is not functioning properly and it stops talking to the server. Users shall be given a chance to retest failed operations when they are able to reconnect

5.10.7 Database Error Handling

The system shall gracefully recover from any failure to connect to the database; and, the system shall not allow for transactions to be incomplete thereby damaging the information stored in the database. Rollback mechanisms shall be used whenever operation of the database fails to complete successfully.

5.10.8 General Error Messages

The system should provide easy to understand error messages without revealing sensitive implementation details and internal system information. All major errors will be logged in the system for debugging, maintenance and future system enhancement.

5.11 Business Rules

These business rules provide operational policies for user access, content submission, analysis execution, data ownership, and security in the DeepGuard seystem.

5.11.1 User Access Rules

Access to protected features of the DeepGuard shall be allowed only to registered and authenticated users of the system. Unregistered users will not be allowed to access the content analysis, submission history, and other protected application features.

5.11.2 Role-Based Access Rules

The system shall be able to recieve, analyze, and add textual content and images from standard users, and provide functionality to allow the user to view his/her analysis history and manage personal accounts. Administrative User will be able to configure the system, observe system activities and do administrative tasks. Administrator privileges will only be granted to authorized administrator accounts.

5.11.3 Data Ownership Rules

The authenticated users will only be able to access their own content, analysis history and stored prediction results. The system shall not allow users to see, alter, or delete data of other users.

5.11.4 File Upload Rules

Content shall not be allowed to be submitted for analysis until successful authentication. Text and supported images formats (JPG, JPEG and PNG) shall be entered into the system within the specified size constraints. Uploaded files that are not supported or larger than the permitted file size will not be analyzed.

5.11.5 Analysis Execution Rules

The system shall only analyze user submissions that have been successfully validated. To ensure system consistency and optimal use of resources, user shall not be allowed to submit multiple analysis requests simultaneously for the same submission.

5.11.6 Session Rules

In order to ensure that users' accounts are protected, authenticated user seessions will expire automatically after a period of no activity. Where appropriate, the system shall require users to re-authenticate before performing sensitive account-related operations.

5.11.7 Report Access Rules

The the system shall enable users to only see their own completed analysis history and prediction results. Only after successfully completing the analysis, analysis records will be available.

5.11.8 Security Enforcement Rules

Before granting access to protected resources or sensitive operations the system shall perform authentication and authorization. Automatic blockage of unauthorised attempts to access the system and automatic recording within system audit logs for monitoring and security analysis.

5.12 Detailed Subsystem Design

The DeepGuard system consists of several interconnected subsystems whose work is to analyze the content of social networks and assess whether it is authenttic or misleading. Every subsystem is responsible for a particular function and delivers its output to the Fusion Engine, which creates the final prediction.

5.12.1 Text Analysis Subsystem

The Text Analysis subsystem receives the user inputted text. Uses a transformer-based model and NLP to detect indicators of misinformation, suspicious claims, clickbait language patterns, and misleading language patterns. The subsystem produces a text credibility score and passes it on to the Fusion Engine.

5.12.2 Image Analysis Subsystem

The Image Analysis sub-system is responsible for analysing the images uploaded with the help of the deep learning computer vision techniques. It is able to capture visual characteristics and detect indications of alterations, synthetic ‘creation’ or deceptive visual information. The subsystem is used to generate an image authenticity score to be used in subsequent processing.

5.12.3 Context Consistency Subsystem

If both text and visual information are present, the Context Consistency subsystem is used to compare the two types. It compares statement found in the text to the content depicted in the image. All discrepancies that are identified are considered a possible sign of misinformation and are incorporated into the final judgment.

5.12.4 Sentiment Analysis Subsystem

The Sentiment Analysis subsystem will analyze the sentiment of the textual data. It identiffies emotionally charged, fear-inducing, urgency-based or manipulative language patterns that are often present in misleading posts. The sentiment score generated is then used as an additional piece of supporting evidence for final decision making.

5.12.5 Metadata Analysis Subsystem

When available, the Metadata Analysis subsystem extracts and analyzes the metadata of the images, including creation date, device information, and editing history. The subsystem detects anomalies that could mean alteration of the image, or that it has been misused, and it supplies corroborating evidence.

5.12.6 Viral Pattern Detection Subsystem

The Viral Pattern Detection subsystem detects common patterns of spreading misinformation such as chain messages, clickbait wording, urgency wording and repetitive forwarding patterns. The subsystem identifies risks by analyzing the characteristics of the viruses.

5.12.7 Fusion Engine Subsystem

The Fusion Engine is the core decision making unit of DeepGuard. It collects the result of all available analysis modules and combines them according to predefined decision logic. The subsystem evaluates the entire body of evidence, determining its likelihood of being fake or authentic.

5.12.8 Output Generation Subsystem

The Output Generation subsystem shows the user the final results. It shows the prediction result, its confidance score and a short description of the reasons for the decision. The subsystem guarantees that the results of analyses are conveyyed clearly and understandably.

5.12.9 Admin Dashboard Subsystem

The Admin Dashboard subsystem lets authorized administrators view system statistics, monitor analysis activities, view reports created, and manage system operations. This subsystem is only accessible via administrator authentication mechanisms.

5.13 System Design Diagrams

The following diagrams are for the structural, behavioural and data flow characteristics of the DeepGuard system. These diagrams present pictures of system functions, relationships between components, movement of data, and relationships between subsystems.

5.13.1 UseCase Diagram

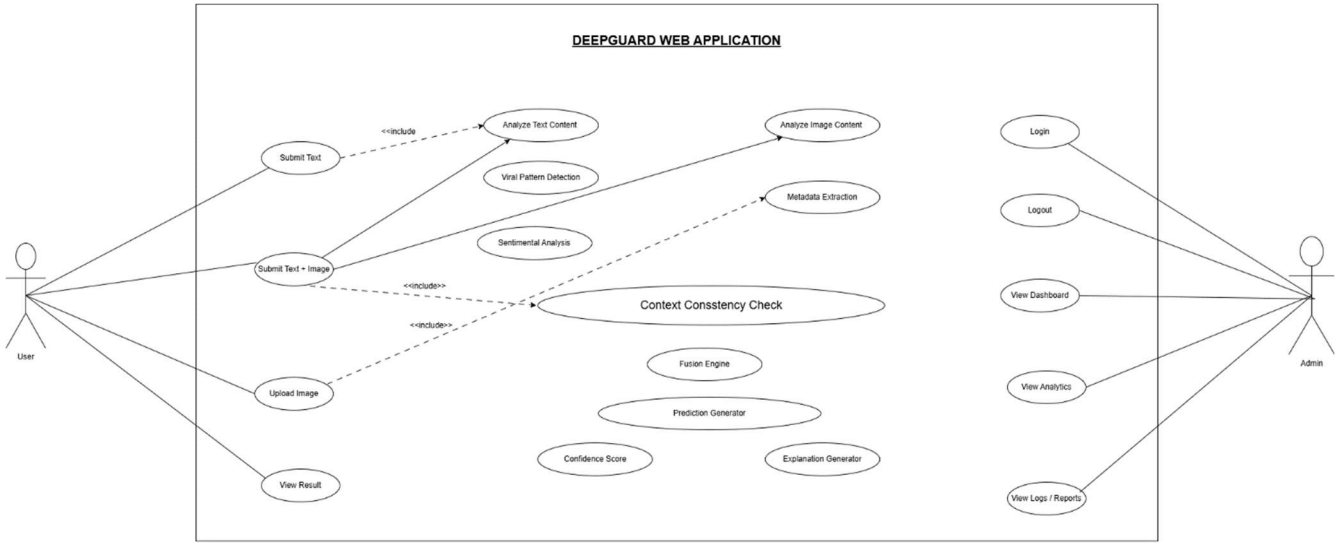


Figure 5.1. UseCase Diagram of DeepGuard System

This usecase diagram shows two actorz involved in the system: the User and thee Administrator. A regular user can doon submit text or image content, run the analyasis, and check their previmous results. The admin covers managging the system and keeping track of the overall usage. It gives a straightforwward view of who does and whatis in thiz in DeepGuard.

13.2 Erd Diagram

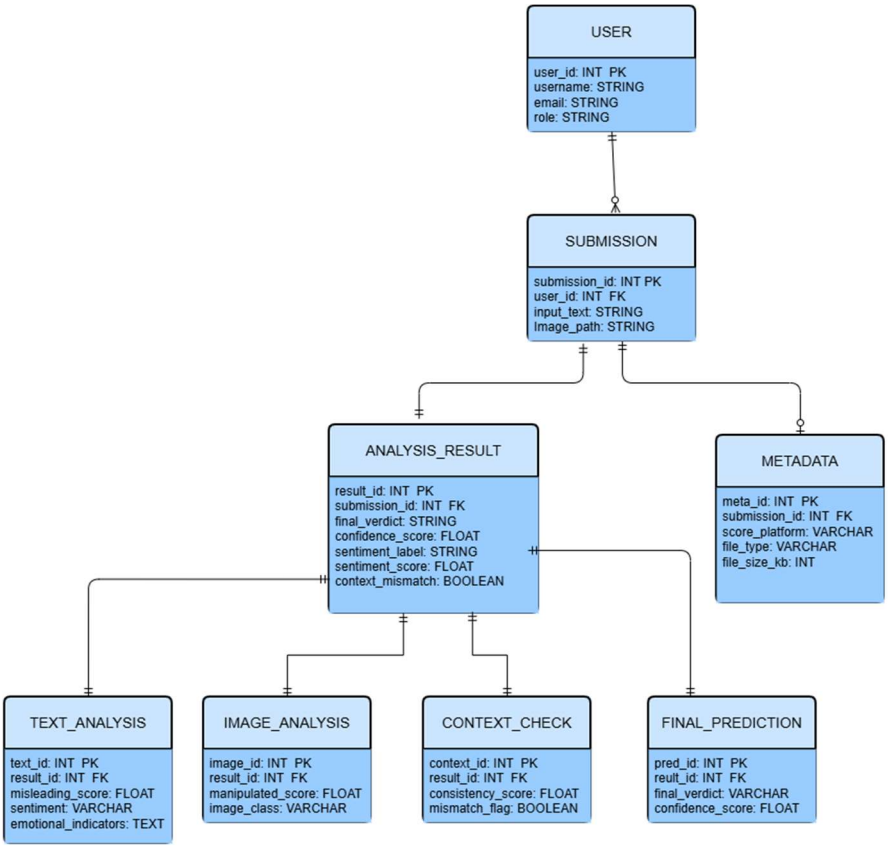


Figure 5.2. Entity Relationship Diagram of the DeepGuard System

The Entity Relationship Diagranm maps out the database structture behind the DeepGuard. It shows thee main entities: User, Submisssion, and Analyssis Result and how they are linked. Each submission belongs to a user, and eeach analysis result ties back to a specific submission. The diagram wass used as a foundaytion when building the SQLite database.

5.13.3 Architecture Diagram

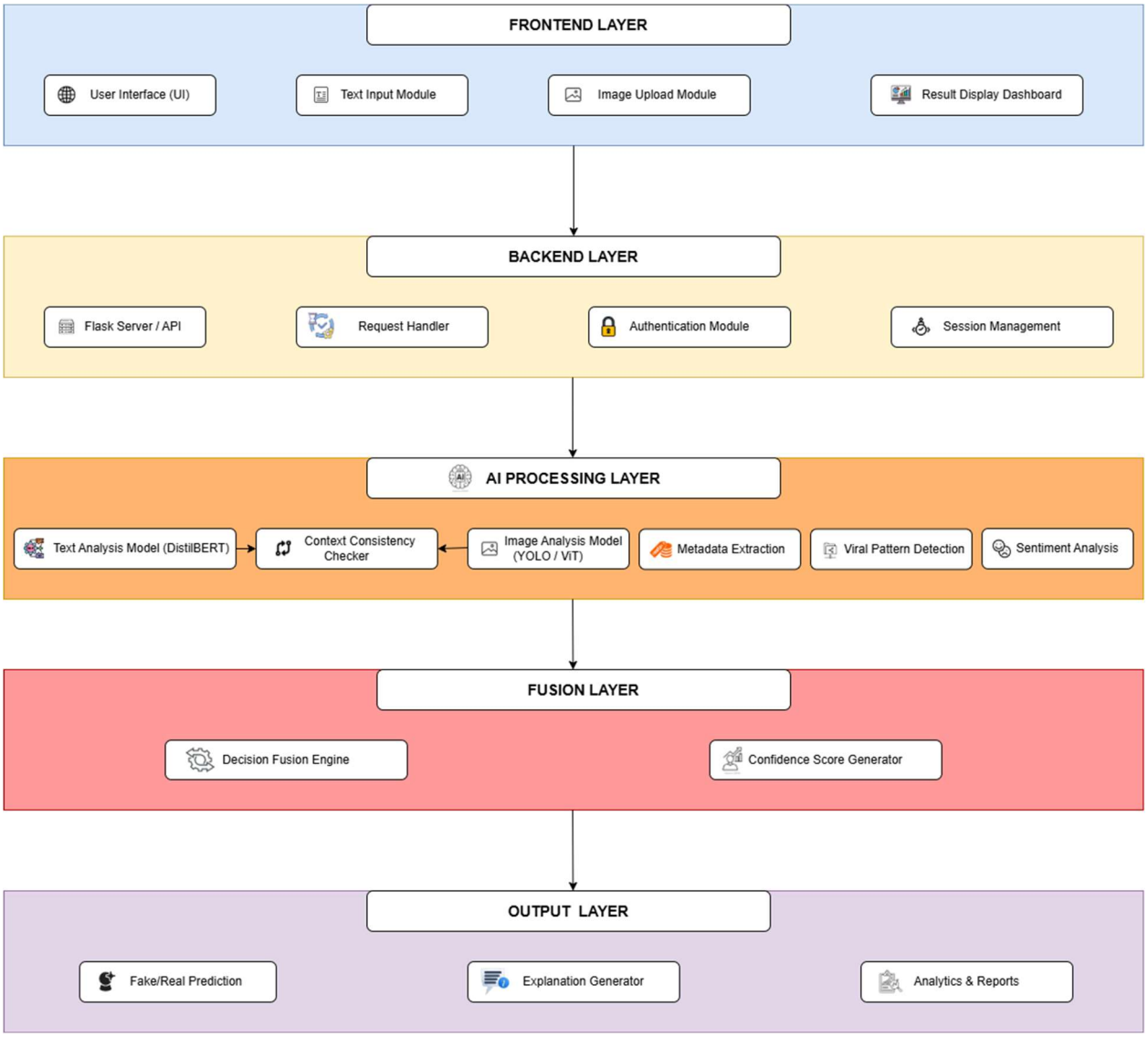


Figure 5.3. Layered Architecture Diagram of the DeepGuard System

The Architecturaal diagram breaks the system into its core layers. The frnt layer is the HTML/CSS/JavasScript interface where the user submits content. Behind it sits the Flassk backend, which routes inputs to the appropriate analysis modules. All module outputs eventuually reach the fusion engine, which calculates the final verdict. The diagram shows how thesse layers stack and communicates.

5.13.4 ACTIVITY DIAGRAM

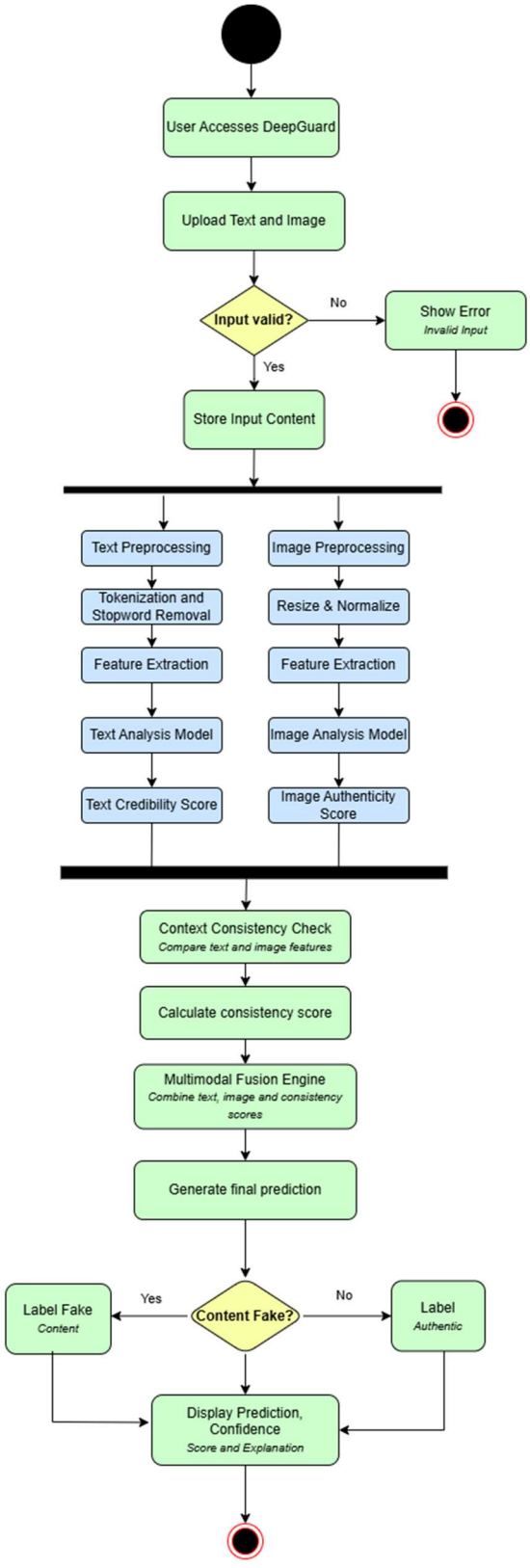


Figure 5.4. Activity Diagram for the DeepGuard Content Analysis Process

This Activity Diagramm walks through what happens from the momment a user submits content to when the result appears on screen. It covers input validation, parallel proccessing across modules, and result fuusion. Where only one input type is provided, the diagram shows which modules get skipped and howz the system continues without them.

5.13.5 Sequence Diagram

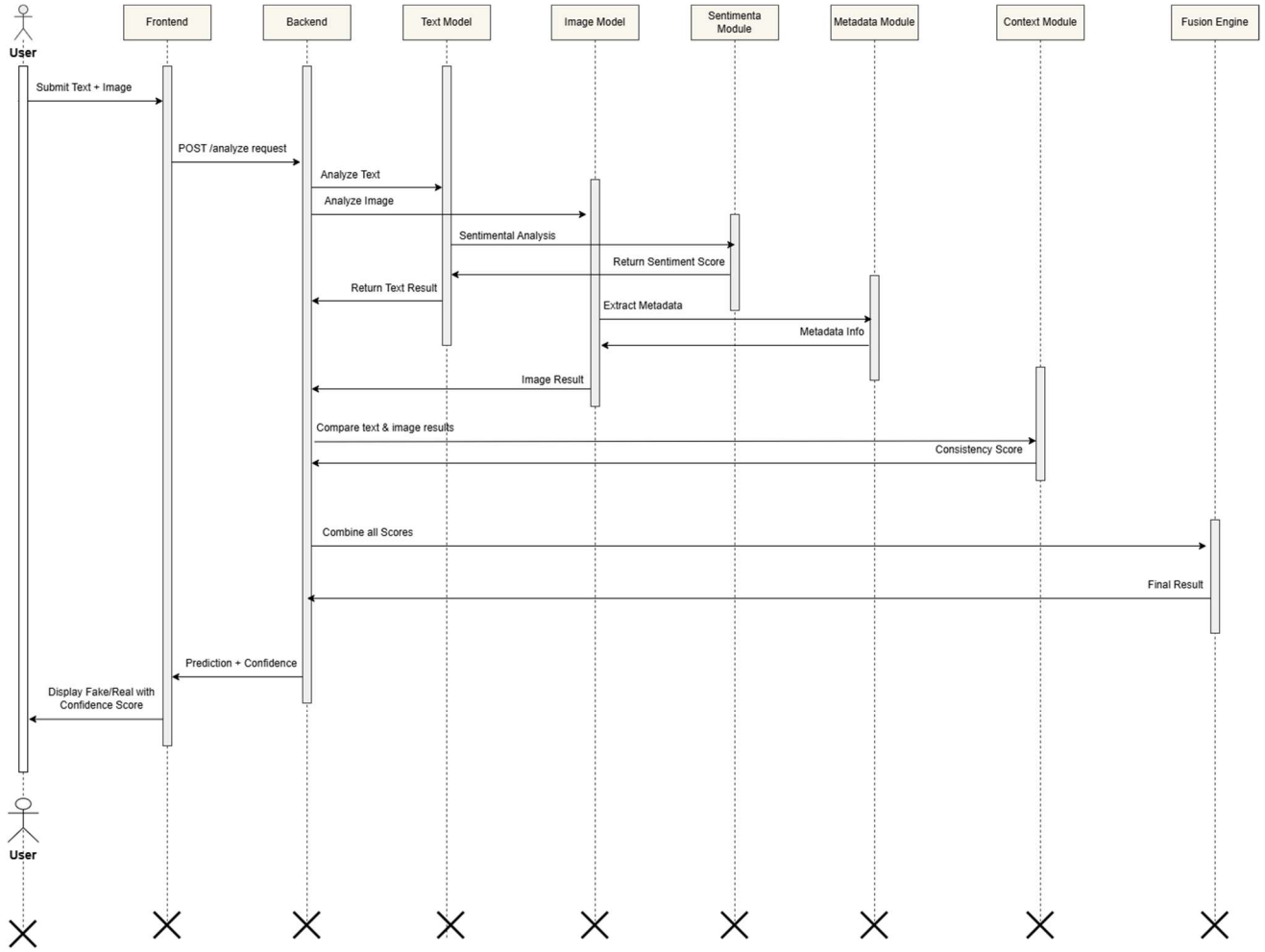


Figure 5.5. Sequence Diagram of the DeepGuard System

This Sequence Diagram shows the order of commmunication among the user, the HTML/CSS/JavaaScript, the Flask backend, the individual analysias modules, and the fuusion engine. It makees clear which component sends a message to which other component, and in what order, during a single analysiss request from start to finish.

5.13.6 Component Diagram

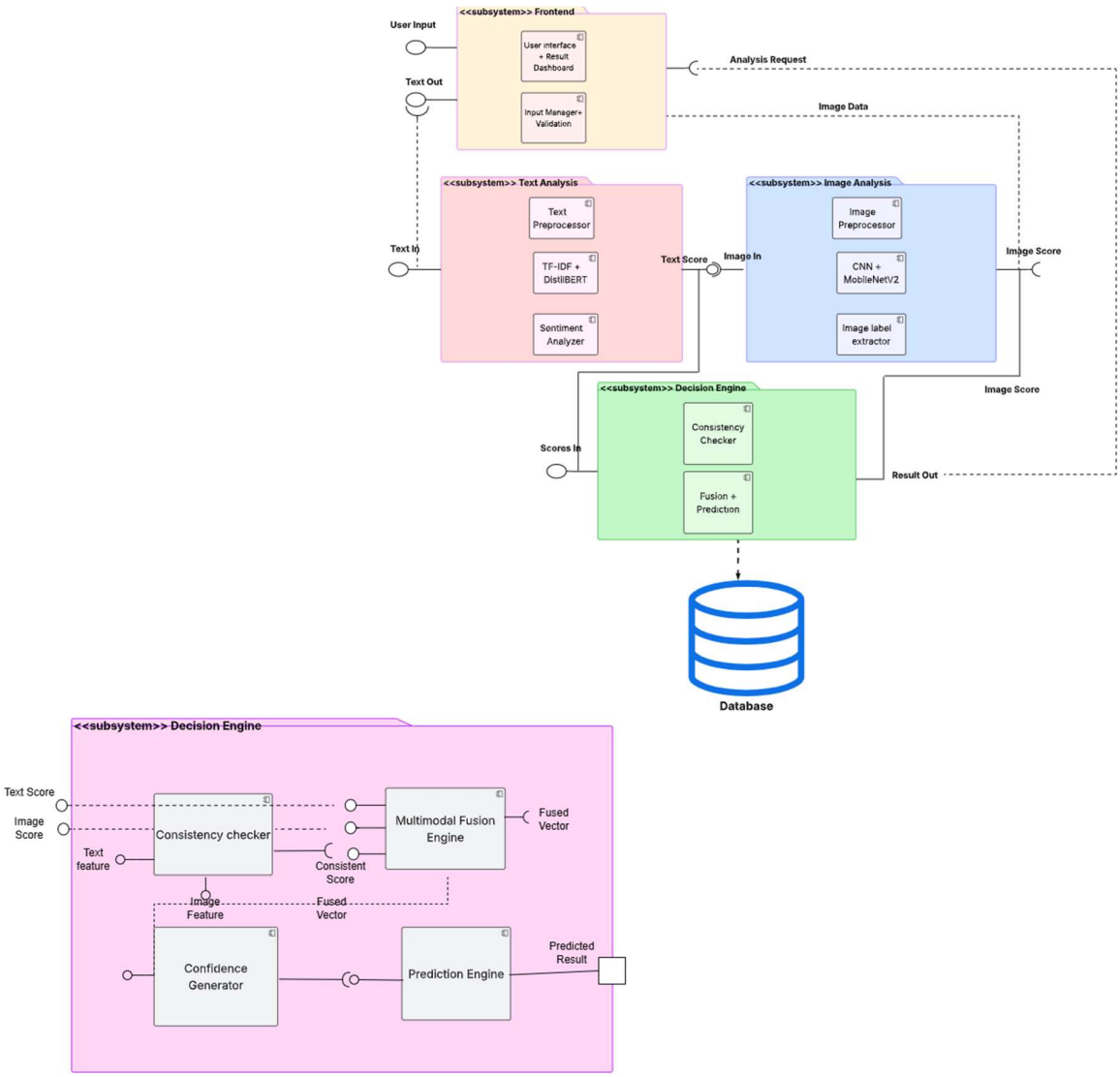


Figure 5.6. Component Diagram of the DeepGuard System

This Component Diagraam lays out all the major software parts of DeepGuard and how theey are connected. The texxt, image, sentiment, metadata, and viral pattern modules are shown as separate compponents, each responsible for a specific type of analysis. Their outputs are passed to the Decision Engine, which is the centrral component that weighs all the incoming scores and determines the final fake or authentic verdict. The frontend interfrace and the SQLite dataabase are shown as distinct components connected to Flask backnd.

5.13.7 State Machine Diagram

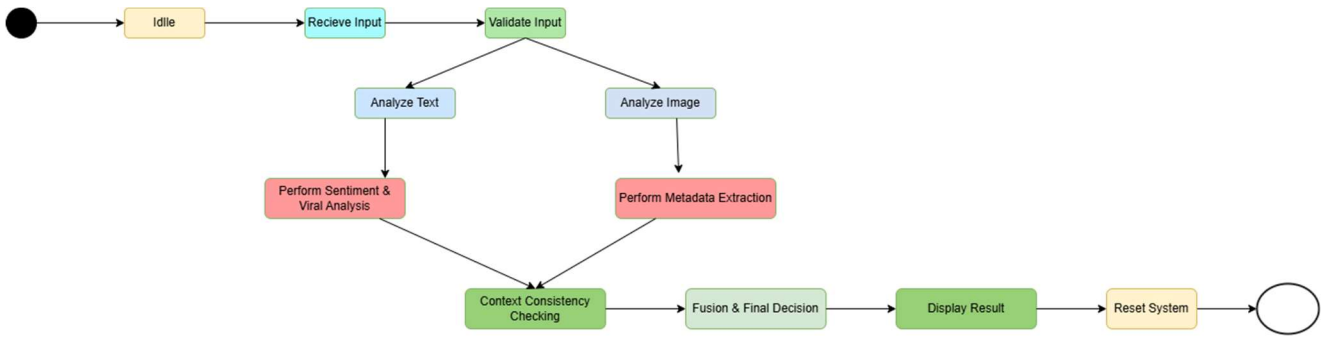


Figure 5.7. State Machine Diagram of the DeepGuard System

This State Machine Diagram follows the different states a content submiission passes through inside the system. It begins at an idle state, shifts to processing once analysis beggins, moves to a completed state when the resulsts are ready, and falls into an error state if something fails. Each transition is triggered by a specific event within of thr system.

5.13.8 Class Diagram

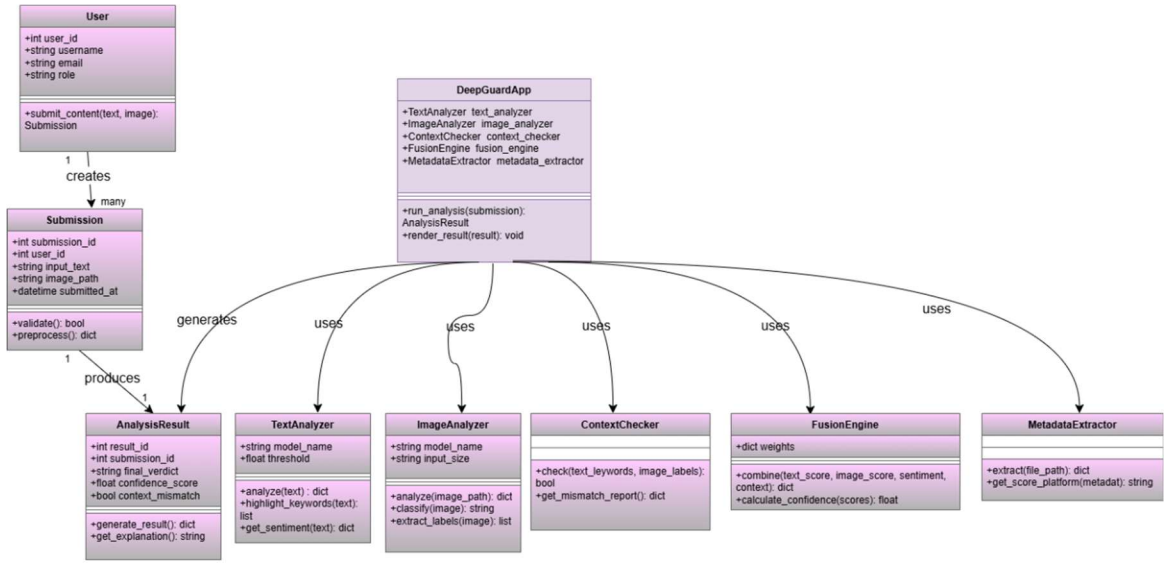
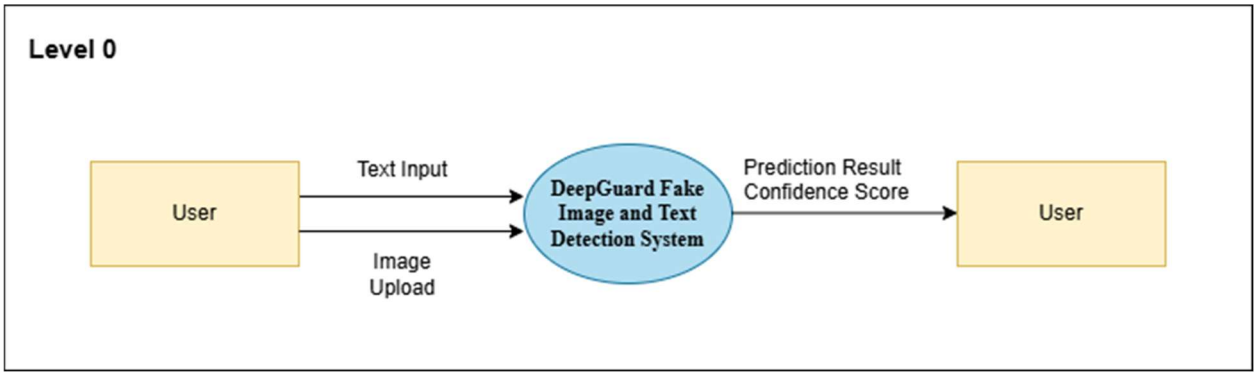


Figure 5.8. Class Diagram of the DeepGuard System

This Class Diagrram shows the main classes that make up DeepGuard's codeebase, including their attributes and methods. It also shows how they are related for innstance; a Submission object hold references to its corrersponding TextAnalysis and ImageAnalysis results. This was thn used during design to plan the strructure before coding begins.

5.13.9 Dataflow Diagram



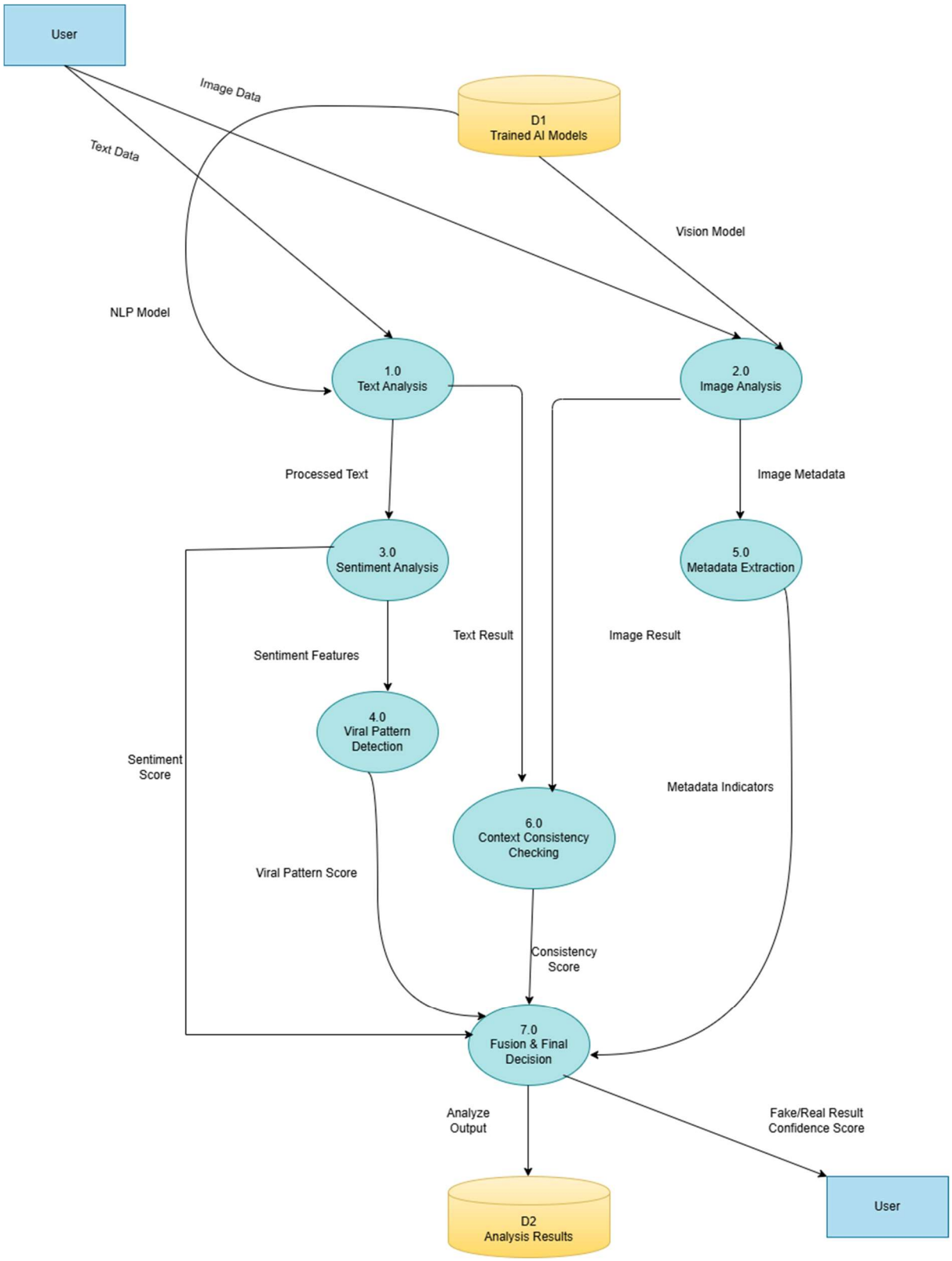


Figure 5.9. Data Flow Diagram of the DeepGuard System

This Dataflow Diagram traces how data moves through the system. User inputs enter from the top, are split and sent to the relevant modules, and each moduule passes a processed sccore forward. These scores converge at the fusion engine, which produces the final classification and confidence score shown to the user.

5.13.10 Database Diagram

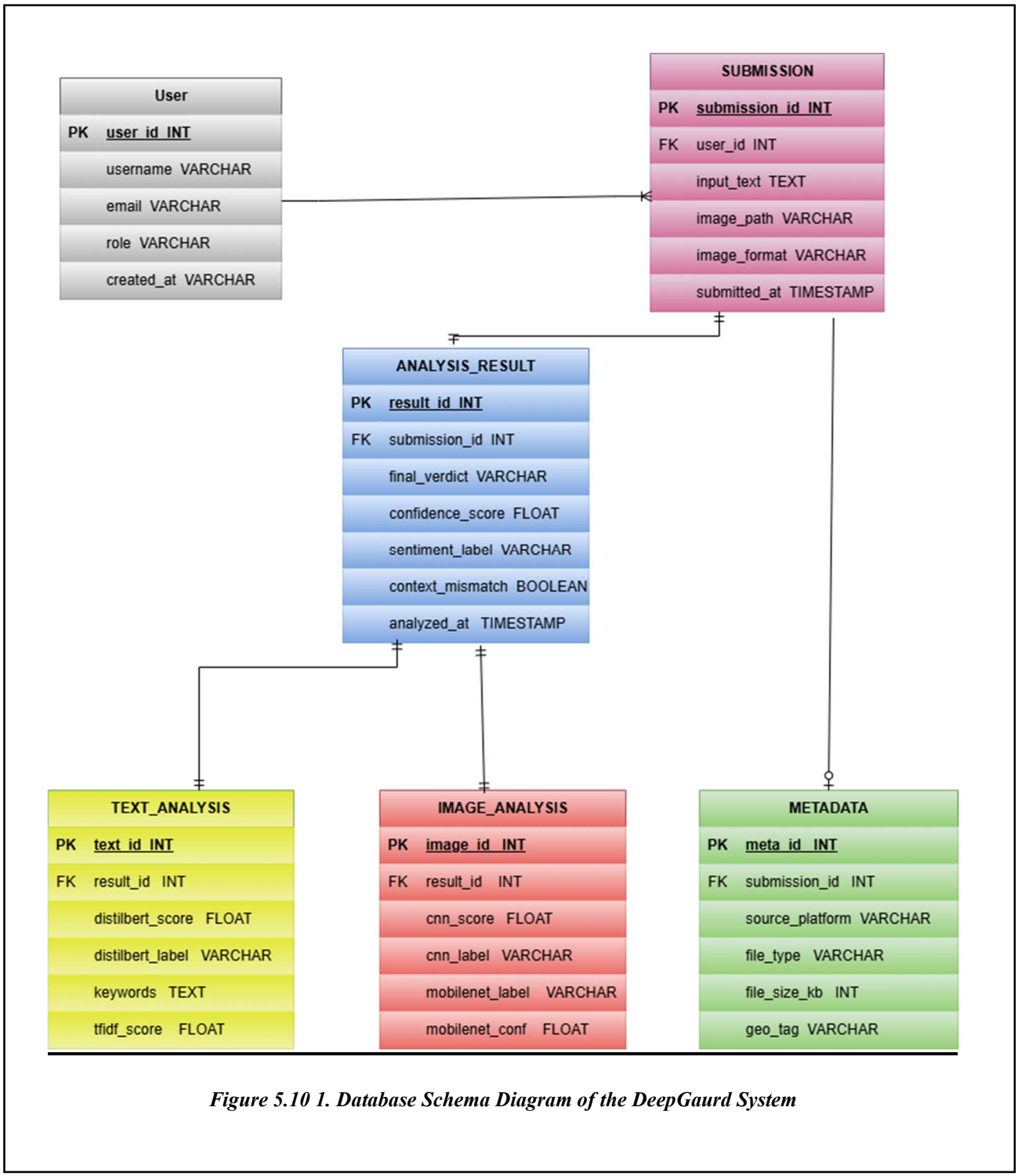


Figure 5.10 1. Database Schema Diagram of the DeepGaurd System

This Daatabase Diagram shows the actual table structuire in DeepGuard's SQLte database. The siix tables-User, Submission, Analysis Result, Text Analysis, Image Analysis, and Metadata - are shownn with their columns and the foreign key relationships connecting themm. It served as the reference when writing the SQL schema during development.

5.14 References

[1] Chikh, A., & Aldayel, M. (2012, December). A new traceable software requirements specification based on IEEE 830. In 2012 International Conference on Computer Systems and Industrial Informatics (pp. 1-6). IEEE.

[2] Schneider, F., & Berenbach, B. (2013). A literature survey on international standards for systems requirements engineering. Procedia Computer Science, 16, 796-805.

[3] Ahmad, I., Yousaf, M., Yousaf, S., & Ahmad, M. O. (2020). Fake news detection using machine learning ensemble methods. Complexity, 2020(1), 8885861.

[4] Wu, L., Morstatter, F., Carley, K. M., & Liu, H. (2019). Misinformation in social media: definition, manipulation, and detection. ACM SIGKDD explorations newsletter, 21(2), 80-90.

[5] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108.

[6] Fields, J., Chovanec, K., & Madiraju, P. (2024). A survey of text classification with transformers: How wide? how large? how long? how accurate? how expensive? how safe?. Ieee Access, 12, 6518-6531.

[7] Diwan, T., Anirudh, G., & Tembhurne, J. V. (2023). Object detection using YOLO: challenges, architectural successors, datasets and applications. multimedia Tools and Applications, 82(6), 9243-9275.

[8] Saravana Ram, R., Vinoth Kumar, M., Al-shami, T. M., Masud, M., Aljuaid, H., & Abouhawwash, M. (2023). Deep Fake Detection Using Computer Vision-Based Deep Neural Network with Pairwise Learning. Intelligent Automation & Soft Computing, 35(2).

[9] Gaind, B., Syal, V., & Padgalwar, S. (2019). Emotion detection and analysis on social media. arXiv preprint arXiv:1901.08458.

[10] Soni, N. (2025). Forensic value of Exif data: an analytical evaluation of metadata integrity across image transfer methods. Perspectives in Legal and Forensic Sciences, 2(2), 10006.

[11] de Freitas Melo, P., Hoseini, M., Zannettou, S., & Benevenuto, F. (2024, May). Don't break the chain: Measuring message forwarding on whatsapp. In Proceedings of the International AAAI Conference on Web and Social Media (Vol. 18, pp. 1054-1067).

[12] Martin Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd Edition, Addison-Wesley, 2003.

[13] Grady Booch, James Rumbaugh, and Ivar Jacobson, *The Unified Modeling Language User Guide*, 2nd Edition, Addison-Wesley, 2005.